

EE6204 Systems Analysis

Beginner Notes: Linear and Nonlinear Programming

Living course note

Updated: 3 September 2026

Scope and sources. This self-contained note develops the supplied Linear Programming and Nonlinear Programming material from `resources/lecture-01-04-linear-and-nonlinear-programming.pdf` (PDF pp. 1–142), with introductory support from `resources/introduction-to-linear-programming.pdf` and teaching emphasis from the Week 1–3 lecture transcripts. The Week 3 exercise class (`resources/week-03-transcript.txt`) and its handwritten worked solutions (`resources/EE6204_Exercises-part1.pdf`) supply the graphical and integer-programming worked examples and the examiner’s marking expectations. Newly archived public examinations and student solutions are not used here until their calculations are independently reviewed.

Contents

1	Optimisation: choosing the best feasible action	1
1.1	How to formulate a real problem	1
2	Linear programming (LP)	2
2.1	Worked formulation: product mix	2
3	The graphical view of an LP	2
3.1	Worked graphical solution	2
4	Preparing an LP for systematic solution	3
5	How the simplex method moves between corners	3
6	Duality and sensitivity: prices hidden inside constraints	4
7	Integer linear programming	5
7.1	Small problems: the integer lattice on the graph	5
7.2	Larger problems: branch and bound	5
8	Structured LPs: transportation and assignment	5
9	Nonlinear optimisation: stationary points are only candidates	6
10	Constraints, Lagrange multipliers, and KKT conditions	6
11	Full-course learning checklist	7

1 Optimisation: choosing the best feasible action

An optimisation problem is a structured way to make a decision. We choose values for *decision variables*, measure their quality using an *objective function*, and enforce limits using *constraints*.

In a general minimisation problem,

$$\min_{x \in \mathbb{R}^n} f(x) \quad \text{subject to} \quad x \in \mathcal{F},$$

where $\mathcal{F}$ is the *feasible set*: the choices that satisfy every constraint. A numerical answer is not a valid solution if it violates a capacity, demand, safety, or non-negativity condition.

1.1 How to formulate a real problem

Start in words and make each modelling choice explicit.

1. Define each variable, including units: for example, “ x = number of suits produced per week.”
2. State whether the objective is to maximise a benefit (profit, throughput) or minimise a cost, time, or waste.
3. Translate each physical condition into an equation or inequality.
4. State the domain. Production amounts usually require $x \geq 0$; whole objects may require $x \in \mathbb{Z}$.

Check signs against the wording: “at most” gives $\leq$; “at least” gives $\geq$. A cost-minimisation model must also include a reason to produce. Otherwise, producing nothing can be the correct mathematical minimum but the wrong operational model.

2 Linear programming (LP)

An LP has a linear objective and linear constraints. A standard maximisation form is

$$\max c^\top x \quad \text{subject to} \quad Ax \leq b, \quad x \geq 0.$$

“Linear” means a variable appears only to the first power with a constant coefficient. Terms such as x_1x_2 , x_1^2 , or x_1/x_2 make a model nonlinear.

2.1 Worked formulation: product mix

Suppose x suits and y gowns earn 30 and 50 units of profit. If material stocks impose three resource limits, the model is

$$\begin{aligned} \max \quad & 30x + 50y \\ \text{s.t.} \quad & 2x + y \leq 16, \\ & x + 2y \leq 11, \\ & x + 3y \leq 15, \\ & x, y \geq 0. \end{aligned}$$

Every coefficient must have a physical interpretation. In the first constraint, 2 is the amount of the first material used per suit, 1 the amount per gown, and 16 the available amount. The last two inequalities exclude negative production; they are constraints too.

3 The graphical view of an LP

With two variables, a linear equality is a straight line and a linear inequality keeps one side of that line. The feasible region is the intersection of all allowed half-planes. To draw it, replace each inequality by equality to obtain a boundary, use a test point to choose the permitted side, and retain only points satisfying every constraint.

The objective has level lines $c^T x = z$. Slide such a line in the improving direction until it makes its last contact with the feasible region. For a finite optimum, a corner (also called an extreme point) attains an optimum. This is why evaluating feasible corners solves a two-variable LP.

Recognise three special outcomes:

- **Infeasible:** no point satisfies all constraints simultaneously.
- **Unbounded:** one can improve the objective forever along a feasible direction.
- **Multiple optima:** the objective line is parallel to an optimal feasible edge, so every point on that edge has the same best value.

Graphing gives valuable intuition but does not scale to many variables, where the feasible set is a high-dimensional polyhedron.

3.1 Worked graphical solution

Consider

$$\begin{aligned} \max \quad & 200x + 550y \\ \text{s.t.} \quad & 100x + 250y \leq 5000, \\ & x + y \leq 50, \quad x, y \geq 0. \end{aligned}$$

Draw each boundary line from two intercepts: $100x + 250y = 5000$ passes through $(50, 0)$ and $(0, 20)$; $x + y = 50$ passes through $(50, 0)$ and $(0, 50)$. The origin satisfies both “ $\leq$ ” inequalities, so the feasible region is the triangle with corners $(0, 0)$, $(50, 0)$, and $(0, 20)$ (the two lines meet at $(50, 0)$, so $x + y \leq 50$ never binds elsewhere). Evaluate the objective at each corner:

$$(0, 0): 0, \quad (50, 0): 10000, \quad (0, 20): 11000.$$

The optimum is $(x, y) = (0, 20)$ with value 11000. Equivalently, the gradient $(200, 550)$ points up and to the right; sliding the level line $200x + 550y = z$ that way, its last contact with the triangle is the vertex $(0, 20)$. In an examination answer, draw and label every constraint line, shade the region, draw one value line with its gradient arrow, and mark the last-contact vertex explicitly — the marker counts the constraint lines and looks for the value line and contact point.

Source: [resources/EE6204_Exercises-part1.pdf](#), Q4 (problem statement only; the solution here is worked for these notes); marking expectations from [resources/week-03-transcript.txt](#).

4 Preparing an LP for systematic solution

Simplex methods use equality constraints and nonnegative variables. The following conversions preserve the feasible choices when the new variables are included:

$$\begin{aligned} a_i^T x \leq b_i & \implies a_i^T x + s_i = b_i, \quad s_i \geq 0 && \text{(slack variable),} \\ a_i^T x \geq b_i & \implies a_i^T x - e_i = b_i, \quad e_i \geq 0 && \text{(surplus variable),} \\ x_k \text{ free} & \implies x_k = x_k^+ - x_k^-, \quad x_k^+, x_k^- \geq 0. \end{aligned}$$

A slack variable measures unused capacity: if $2x + y + s = 16$, then $s = 0$ means the resource is fully used. A surplus variable measures the amount by which a lower-bound requirement is

exceeded. These are not arbitrary algebraic tricks; they give an inequality's missing amount an explicit nonnegative meaning.

5 How the simplex method moves between corners

The graphical method suggests the simplex idea: a finite LP optimum can be found at a corner, so move between adjacent corners instead of searching every point. After converting constraints to equality form, choose a set of basic variables whose columns form an invertible basis. Set all nonbasic variables to zero and solve for the basic variables. The result is a *basic feasible solution* only when every basic variable is nonnegative.

In a tableau, one pivot performs a controlled exchange:

1. Choose an entering variable whose reduced cost indicates improvement under the tableau's sign convention.
2. For rows with a positive entering-column coefficient, compute the nonnegative ratios b_i/a_{ij} . The smallest ratio identifies the leaving variable; this prevents loss of feasibility.
3. Scale the pivot row and eliminate the other entries in the pivot column.
4. Stop when no reduced cost offers improvement. If no row is eligible for the ratio test, the objective is unbounded in that direction.

Never memorise “most positive” or “most negative” without checking how the objective row was defined. The invariant is clearer: the entering variable must improve the objective, and the leaving variable must be the first basic variable that would otherwise become negative.

Ratio-test ties and degeneracy. When the minimum ratio is attained in several rows at once, any of them may be the pivot row; the others then produce basic variables equal to zero. A basic feasible solution with one or more basic variables at zero is *degenerate* — geometrically, a corner where more constraint boundaries meet than the two needed to pin a point in the plane. For example,

$$\max 5x_1 + 7x_2 \quad \text{s.t.} \quad 4x_1 + 5x_2 \leq 20, \quad 2x_1 + 6x_2 \leq 24, \quad 6x_1 + 4x_2 \leq 16, \quad x_1, x_2 \geq 0$$

has all three constraint lines through $(0, 4)$: $5(4) = 20$, $6(4) = 24$, $4(4) = 16$. Entering x_2 , the ratio test gives $20/5 = 24/6 = 16/4 = 4$ — a three-way tie. Whichever row is pivoted, the optimal tableau has $x_2 = 4$, objective 28, and the other two slacks stuck at zero. Recognising a degenerate tableau (a ratio-test tie, or a zero in the basic-variable column at optimality) is a common short-answer target.

Source: resources/EE6204_Exercises-part1.pdf, Q1–Q2; resources/week-03-transcript.txt. The PDF's gradient annotation $\nabla f = [7; 5]$ is transposed; for $f = 5x_1 + 7x_2$ it is $[5; 7]$, which does not change the answer..

Equality and $\geq$ rows may lack an obvious starting basis. An artificial variable supplies a temporary identity column. In the two-phase method, Phase I minimises the sum of artificial variables. A positive Phase I optimum proves that the original constraints are infeasible; otherwise remove the artificials and optimise the true objective in Phase II.

Source: resources/lecture-01-04-linear-and-nonlinear-programming.pdf, PDF pp. 16–59..

6 Duality and sensitivity: prices hidden inside constraints

For the pair

$$\min c^\top x \quad \text{s.t. } Ax \geq b, x \geq 0, \quad \max b^\top w \quad \text{s.t. } A^\top w \leq c, w \geq 0,$$

the dual variable w_i measures the marginal value of the corresponding primal requirement, within an allowable perturbation range. Weak duality says every feasible dual value bounds every feasible primal value. At finite optima, strong duality makes the two values equal.

Using the dual to make a problem graphable. The dual has exactly one variable per primal constraint and one constraint per primal variable. So a primal with many variables but only two constraints has a dual with only two variables, which can be solved graphically. If a problem is posed with more than two variables and the examiner still asks for a graphical solution, form the dual, solve the two-variable dual on paper, and recover the primal optimum from complementary slackness: a primal variable is zero wherever its dual constraint is slack, and a primal constraint is tight wherever its dual variable is positive. Count variables against constraints first, then graph whichever of the primal or dual has two variables.

Source: resources/week-03-transcript.txt (lecturer's stated quiz technique: "if I request you to use a graphic way ... you have to use the duality theory to transform this into the dual problem, [which] will have a fewer number of variables").

Complementary slackness explains which quantities can be positive:

$$w_i((Ax)_i - b_i) = 0, \quad x_j(c_j - (A^\top w)_j) = 0.$$

A slack resource constraint therefore has zero shadow price; a positive activity makes its dual constraint tight. This is a logical either-or statement, not permission to divide by a term that might be zero.

If an optimal basis B is kept after b changes to $b + \Delta b$, its new basic solution is $B^{-1}(b + \Delta b)$. The old basis remains feasible exactly while this vector is nonnegative. Objective-coefficient changes instead require the reduced costs to retain their optimal signs. Outside an allowable range, re-optimize rather than extrapolating.

Source: resources/lecture-01-04-linear-and-nonlinear-programming.pdf, PDF pp. 60–90..

7 Integer linear programming

If some variables must take whole-number values (numbers of products, vehicles, or people), the model is an *integer linear program* (ILP). Dropping the integrality requirement gives the *LP relaxation*. For a maximisation, the relaxation's optimal value is an upper bound on the ILP's optimal value, but rounding a fractional relaxed solution can be infeasible or far from optimal, so rounding is not a valid method.

7.1 Small problems: the integer lattice on the graph

With two integer variables, solve graphically. Draw the feasible region of the relaxation, then consider only its lattice points (integer coordinates). Slide the objective level line in the improving direction until its last contact with a feasible lattice point; that point is the ILP optimum. Show the value line and the contact construction, exactly as in the continuous graphical method — listing every lattice point and picking the best by inspection is treated as showing no method.

7.2 Larger problems: branch and bound

Branch and bound searches without full enumeration. Solve the LP relaxation. If its optimum is already integer, stop. Otherwise choose a variable x_k with fractional value f and split into two subproblems: add $x_k \leq \lfloor f \rfloor$ to one and $x_k \geq \lceil f \rceil$ to the other. Solve each relaxation and recurse. Prune a branch when it is infeasible, when its relaxation value is no better than the best integer solution found so far (the *incumbent*), or when it produces an integer solution (then update the incumbent).

Worked example.

$$\begin{aligned} \max \quad & 7x_1 + 5x_2 \\ \text{s.t.} \quad & 4x_1 + 6x_2 \leq 18, \quad 2x_1 + 6x_2 \leq 9, \quad 6x_1 + 3x_2 \leq 27, \quad x_1, x_2 \geq 0 \text{ integer.} \end{aligned}$$

The LP relaxation optimum is $(x_1, x_2) = (4.5, 0)$ with value 31.5. Branch on x_1 :

- $x_1 \geq 5$: infeasible ($6x_1 \leq 27$ forces $x_1 \leq 4.5$). Prune.
- $x_1 \leq 4$: relaxation optimum $(4, \frac{1}{6})$, value $\frac{173}{6} \approx 28.83$. Branch on x_2 :
 - $x_2 \leq 0$: optimum $(4, 0)$, value 28, integer. Set incumbent = 28.
 - $x_2 \geq 1$: relaxation optimum $(1.5, 1)$, value 15.5 < 28. Prune.

The optimal integer solution is $(x_1, x_2) = (4, 0)$ with value 28.

Source: [resources/EE6204_Exercises-part1.pdf](#), Q3 (the source branch-and-bound tree is left unfinished; the branches above are completed for these notes); [resources/week-03-transcript.txt](#). Branch and bound is not in [resources/lecture-01-04-linear-and-nonlinear-programming.pdf](#); the lecturer introduced it verbally as “the end of the LP.” For Quiz 1, expect the graphical integer-lattice method rather than a full branch-and-bound tree; branch and bound itself is final-exam material..

8 Structured LPs: transportation and assignment

In a transportation model, x_{ij} is the amount shipped from source i to destination j :

$$\min \sum_{i,j} c_{ij}x_{ij}, \quad \sum_j x_{ij} = s_i, \quad \sum_i x_{ij} = d_j, \quad x_{ij} \geq 0.$$

Balance total supply and demand, adding a clearly interpreted dummy source or destination if necessary. Starting from a basic allocation, calculate row and column potentials from $u_i + v_j = c_{ij}$ on basic cells. A negative reduced cost $c_{ij} - u_i - v_j$ identifies an improving nonbasic cell for minimisation. Add it, trace a closed alternating $+/-$ loop, and shift the largest amount allowed by the cells marked minus.

The assignment problem is a one-to-one special structure. The Hungarian method subtracts row minima, then column minima, and seeks independent zeros. When too few independent zeros exist, cover all zeros with the minimum number of lines, adjust uncovered and doubly covered entries, and repeat. These reductions preserve all complete-assignment comparisons because every complete assignment uses exactly one element from each row and column.

Source: [resources/lecture-01-04-linear-and-nonlinear-programming.pdf](#), PDF pp. 65–90..

9 Nonlinear optimisation: stationary points are only candidates

For a differentiable scalar function, Newton’s iteration seeks a stationary point:

$$x_{k+1} = x_k - \frac{f'(x_k)}{f''(x_k)}.$$

It is fast near a regular solution but can fail when f'' is small, the starting point is poor, or the stationary point is not the desired optimum. Always classify the result. In several variables, solve

$$H(x_k)p_k = -\nabla f(x_k), \quad x_{k+1} = x_k + p_k,$$

where $H = \nabla^2 f$. At a stationary point, $H \succ 0$ certifies a strict local minimum, $H \prec 0$ a strict local maximum, and an indefinite Hessian a saddle. These are local claims unless convexity supplies a global guarantee.

Derivative-free interval searches use a different construction. Fibonacci and golden-section search repeatedly discard a portion of a unimodal interval while reusing one previous function value. They return a bracket with stated uncertainty, not an exact optimizer.

Source: [resources/lecture-01-04-linear-and-nonlinear-programming.pdf](#), PDF pp. 97–128..

10 Constraints, Lagrange multipliers, and KKT conditions

For equality constraints $h_i(x) = 0$, define

$$L(x, \lambda) = f(x) - \sum_i \lambda_i h_i(x).$$

At a regular constrained optimum, stationarity $\nabla_x L = 0$ holds together with all original constraints. Geometrically, the objective gradient lies in the span of the active constraint normals: no feasible first-order direction can improve the objective.

For the course convention

$$\min f(x) \quad \text{s.t.} \quad h_i(x) = 0, \quad g_j(x) \geq 0,$$

use $L = f - \lambda^\top h - \mu^\top g$. The Karush–Kuhn–Tucker conditions are

$$\begin{array}{ll} \nabla_x L = 0 & \text{(stationarity),} \\ h_i(x) = 0, \quad g_j(x) \geq 0 & \text{(primal feasibility),} \\ \mu_j \geq 0 & \text{(dual feasibility),} \\ \mu_j g_j(x) = 0 & \text{(complementary slackness).} \end{array}$$

An inactive inequality has $g_j(x) > 0$ and therefore $\mu_j = 0$; an active inequality has $g_j(x) = 0$ and may carry a positive multiplier. KKT conditions alone do not prove a global optimum for a nonconvex problem. With convex objective/feasible geometry and an appropriate regularity condition, they become a global certificate.

Source: [resources/lecture-01-04-linear-and-nonlinear-programming.pdf](#), PDF pp. 120–141..

11 Full-course learning checklist

- Can I identify variables, objective, constraints, and domains in a word problem?
- Can I turn “at most” and “at least” into correctly signed inequalities?
- Can I draw a two-variable feasible region and diagnose infeasibility, unboundedness, or multiple optima?
- Can I add slack or surplus variables and explain their physical meaning?
- Can I explain the entering/leaving logic of one simplex pivot and the purpose of Phase I?
- Can I form a primal–dual pair and use complementary slackness without reversing signs?
- Can I dualise a > 2 -variable, 2-constraint LP so it can be solved graphically?

- Can I solve a two-variable integer LP on the graph, and outline one branch-and-bound step?
- Can I distinguish a stationary point, local optimum, and convex global optimum?
- Can I write all four KKT components using one consistent inequality convention?